REPORT 004: EPOCH 4: HARDWARE LED CONTROLLER, DIRTY-FLAG DIFF COALESCING & 8OPPERC COCKPIT LED MODES

2026-11-04 00:25:00 | Epoch 4 of 20: src/led_controller.rs (28-CC nanoKONTROL2 vs. 24-Note MIDImix LED protocols, Internal/External/Op8Cockpit modes, dirty-flag coalescing, and full-panel hot-plug resync)

Operator: Gemini 4 Argon (Medium) · Follows: report-003

Prompt

```
Epoch 4 of 20: Implement the Hardware LED Controller in src/led_controller.rs
↪  supporting device-specific LED protocols (Korg nanoKONTROL2 28 CC-driven LEDs on
↪  channel 1 vs. Akai MIDImix 24 Note-driven LEDs on channel 1). Support three
↪  operating modes per device: (1) Internal mode (LEDs reflect button
↪  toggle/momentary state), (2) External mode (LEDs driven by incoming DAW/synth
↪  MIDI or OSC feedback), and (3) 8opPerc Cockpit mode (Solo, Mute, Rec, and
↪  Transport LEDs reflect live percussion voice lane states, timbre reroll flashes,
↪  and bar beat indicators). Include dirty-flag diff coalescing so only changed LED
↪  states are transmitted, plus automatic full-panel LED state resynchronization
↪  upon hot-plug reconnection or preset load. Add automated tests (with
↪  CARGO_BUILD_JOBS=2), git commit all changes, and compile reports/report-004.html
↪  from reports/report-004.md via
↪  /home/renick/claude/html-report-generator/build-report.sh --pdf and commit.
```

TL;DR

Implemented `src/led_controller.rs` (`DeviceLedController`, `MultiDeviceLedManager`, `LedOperatingMode`, `Op8CockpitLedSnapshot`, and `TrackedLedEntry`). Supports device-specific hardware LED protocols (Korg nanoKONTROL2's 28 CC-driven LEDs on Channel 1 `0xB0` vs. Akai MIDImix's 24 Note-driven LEDs on Channel 1 `0x90`), three operating modes per device (`Internal`, `External`, and `Op8Cockpit` with live percussion lane Solo/Mute states, timed timbre reroll Rec flashes, and bar downbeat Cycle pulses), dirty-flag diff coalescing so only net-changed LEDs are transmitted over USB MIDI, and automatic full-panel LED resynchronization upon hot-plug reconnection or preset load. Verified by 34 passing automated tests (`28` Rust unit/integration tests + `6` Node.js SDK tests).

1. Plain-Language Overview & Epoch 4 Goals

Physical MIDI controllers have illuminated buttons (LEDs) that tell a musician at a glance which tracks are muted, which tracks are soloed, and whether the sequencer is playing or recording. However, different hardware manufacturers use different MIDI message formats to turn those lights on and off:

- On a **Korg** `nanoKONTROL2`, all **28 button LEDs** (`8 Solo` + `8 Mute` + `8 Rec` + `4 Transport: Play, Stop, Record, Cycle`) are controlled by sending **Control Change (`CC`)** messages on MIDI Channel 1 (`status = 0xB0`, value `127` = ON, `0` = OFF).
- On an **Akai** `MIDI Mix`, all **24 button LEDs** (`8 Mute` + `8 Solo` + `8 Rec Arm`) are controlled by sending **Note On (`Note`)** messages on MIDI Channel 1 (`status = 0x90`, velocity `127` = ON, `0` = OFF).

Furthermore, flooding a USB 1.1/2.0 MIDI cable with redundant LED messages on every GUI frame wastes bandwidth and can introduce jitter into real-time knob and drum triggers. Conversely, when a controller is unplugged and plugged back in (or when a new preset is loaded), the hardware forgets all its lights and needs a complete, one-time full-panel refresh.

In **Epoch 4 of 20**, we implemented `src/led_controller.rs` to solve all three challenges cleanly:

1. **Device-Specific LED Protocol Engine (`DeviceLedController`)**: Automatically encodes 28 CC packets (`0xB0`) for the Korg `nanoKONTROL2` and 24 Note On packets (`0x90`) for the Akai `MIDI Mix` using each device's declarative `LedProtocol`.
2. **Three Operating Modes Per Device (`LedOperatingMode`)**:
   (a) `Internal`: Hardware LEDs mirror the local button `Toggle` / `Momentary` states from `DeviceMappingPipeline`.
   (b) `External`: Hardware LEDs are controlled exclusively by incoming MIDI packets from a DAW/synthesizer or `/nkeditor/led/set` OSC commands.
   (c) `Op8Cockpit`: Hardware LEDs act as a live dashboard for the 8-voice `8opPerc` percussion engine—showing per-lane `Solo` and `Mute` states, timed `Rec` flashes when a voice lane's FM timbre is rerolled, `Play`/`Stop`/`Record` transport status, and bar downbeat pulses on the `Cycle` LED.
3. **Dirty-Flag Diff Coalescing & Full-Panel Resynchronization**: Tracks both `desired_state` and `last_transmitted_state` per LED (`TrackedLedEntry`). If an LED is set to the same value multiple times—or toggled ON and back OFF before the frame flush—zero redundant MIDI packets are sent (`flush_dirty_packets`). On USB hot-plug reconnect (`on_hotplug_reconnect`) or preset load (`on_preset_loaded`), `resync_full_panel` marks all 28 or 24 LEDs dirty and transmits the complete hardware panel state in one clean burst.

Plain-English Concepts Mapped to Code Components

Plain-English Concept	What It Does for the Musician	Exact Code Component
Hardware-Aware LED Encoder	Sends CC `0xB0` for Korg `nanoKONTROL2` (28 LEDs) and Note `0x90` for Akai `MIDI Mix` (24 LEDs)	`src/led_controller.rs` (`DeviceLedController`, `DeviceLedController::encode_led_packet`)
Three LED Operating Modes	Switches between `Internal` button latching, `External` DAW/OSC control, and `Op8Cockpit` live rig telemetry	`src/led_controller.rs` (`LedOperatingMode::{Internal, External, Op8Cockpit}`)
`8opPerc` Cockpit LED Dashboard	Lights Mute/Solo lanes, flashes Rec LEDs on timbre rerolls, and pulses Cycle on bar downbeats	`src/led_controller.rs` (`Op8CockpitLedSnapshot`, `Op8VoiceLedState`, `tick_op8_animations`)
Dirty-Flag Diff Coalescing	Eliminates redundant USB MIDI traffic by transmitting only net-changed LEDs	`src/led_controller.rs` (`TrackedLedEntry`, `DeviceLedController::flush_dirty_packets`)
Full-Panel Hot-Plug Resync	Relights all 28/24 LEDs automatically when a controller is re-plugged or a preset loads	`src/led_controller.rs` (`DeviceLedController::resync_full_panel`, `MultiDeviceLedManager::handle_hotplug_event`)

2. Phase 1: Device-Specific LED Protocols & Three Operating Modes (‘src/led_controller.rs‘)

‘Internal‘, ‘External‘, and ‘Op8Cockpit‘ LED Routing

`DeviceLedController` maintains dual fast-lookup maps (`name_to_index` for dot-notated names like `"track1.solo"` or `"transport.cycle"`, and `number_to_index` for raw CC/Note numbers like `32` or `46`) while preserving the hardware panel order in `leds: Vec`:

- **Mode 1 (`LedOperatingMode::Internal`)**: `handle_internal_button_event` consumes `ButtonStateChangeEvent` values from `DeviceMappingPipeline` and updates the corresponding LED’s `desired_state`. While in `Internal` mode, incoming external MIDI LED messages are ignored so external loopback noise cannot corrupt local button indicators.
- **Mode 2 (`LedOperatingMode::External`)**: Ignores local button presses and instead updates LEDs only when `handle_external_midi_packet` (matching the device’s `LedProtocol` channel and message type) or `handle_external_osc_led` (`/nkeditor/led/set`) is invoked.
- **Mode 3 (`LedOperatingMode::Op8Cockpit`)**: Maps `Op8CockpitLedSnapshot` onto the controller’s physical LEDs: `lanes[0..7].muted` -> `track1.mute .. track8.mute` (`CC 48..55` on nanoKONTROL2, `Notes 1, 4, ..., 22` on MIDImix).
- `lanes[0..7].soloed -> track1.solo .. track8.solo` (`CC 32..39` on nanoKONTROL2, `Notes 2, 5, ..., 23` on MIDImix).
- `lanes[0..7].reroll_flash_ticks > 0 -> track1.rec .. track8.rec` (`CC 64..71` on nanoKONTROL2, `track1.rec_arm .. track8.rec_arm` `Notes 3, 6, ..., 24` on MIDImix), automatically counting down and turning off via `tick_op8_animations()`.
- `playing`, `recording`, and `beat_in_bar == 0` (downbeat) -> `transport.play` (`CC 41`), `transport.stop` (`CC 42`), `transport.record` (`CC 45`), and `transport.cycle` (`CC 46`).

Hardware LED Protocol & Operating Mode Matrix

Controller Model	Total LEDs	MIDI Protocol (‘LedProtocol‘)	‘Internal‘ Mode Source	‘External‘ Mode Source	‘Op8Cockpit‘ Mode Source
Korg nanoKONTROL2	28 (24 track + 4 transport)	`Cc` on Ch `1` (`0xB0`, `127`=ON, `0`=OFF)	`ButtonStateChangeEvent` / `sync_from_pipeline`	Incoming Ch 1 CC / OSC `/nkeditor/led/set`	`Op8CockpitLedSnapshot` (8 lanes + Play/Stop/Rec/Cycle)
Akai MIDI Mix	24 (8 mute + 8 solo + 8 rec_arm)	`Note` on Ch `1` (`0x90`, `127`=ON, `0`=OFF)	`ButtonStateChangeEvent` / `sync_from_pipeline`	Incoming Ch 1 Note / OSC `/nkeditor/led/set`	`Op8CockpitLedSnapshot` (8 lanes Mute/Solo/RecArm flash)

3. Phase 2: Dirty-Flag Diff Coalescing & Full-Panel Resynchronization

Zero-Waste MIDI Flushing and Hot-Plug Recovery

Each `TrackedLedEntry` stores `desired_state: bool`, `last_transmitted_state: Option`, and `dirty: bool`:

- **Dirty-Flag Diff Coalescing (`TrackedLedEntry::set_desired & flush_dirty_packets`)**: Whenever code sets an LED’s state, `dirty` is computed as `self.last_transmitted_state != Some(self.desired_state)`. Setting an already-ON LED to `true` five times in a row increments `coalesced_updates_skipped` and emits only 1 packet on flush. Even better, if an LED is toggled `false -> true -> false` within the same update cycle

before `flush_dirty_packets()` runs, `desired_state` matches `last_transmitted_state` (`Some(false)`), clearing `dirty = false` so **zero** MIDI bytes are transmitted.

- **Full-Panel Resynchronization (`resync_full_panel`, `on_hotplug_reconnect`, `on_preset_loaded`)**: Sets `last_transmitted_state = None` and `dirty = true` across all LEDs on the device and flushes the full 28-packet (nanoKONTROL2) or 24-packet (Akai MIDImix) burst. `MultiDeviceLedManager::handle_hotplug_event` hooks directly into `HotplugEvent::ConnectedNew` and `HotplugEvent::ReconnectedRemembered` so any controller re-plugged into any USB port immediately relights its entire panel.

Tests

- `CARGO_BUILD_JOBS=2 cargo test` — 28 Rust unit and integration tests passing (`4` new test suites in `led_controller::tests` verifying 28-CC nanoKONTROL2 vs 24-Note MIDImix protocols, dirty-flag diff coalescing and oscillation cancellation, `Internal/External/Op8Cockpit` modes with timed timbre reroll flashes and bar downbeat Cycle pulses, and full-panel hot-plug resynchronization), 0 failures
- `npm test` (`node --test js/*.test.js`) — 6 Node.js SDK unit tests passing, 0 failures

QA

```
`34 tests passing (28 Rust + 6 Node.js)`

`28-CC Korg nanoKONTROL2 (0xB0) and 24-Note Akai MIDImix (0x90) LED protocols verified`

`Dirty-flag diff coalescing & oscillation suppression verified`

`Internal, External, and Op8Cockpit modes + 28-LED hot-plug resync verified`

`0 warnings, 0 failures`
```

All Epoch 4 deliverables compiled cleanly with `CARGO_BUILD_JOBS=2` and passed all Rust and Node.js test suites.

Files touched

- ‘src/led_controller.rs‘ — Implemented ‘LedOperatingMode‘, ‘Op8VoiceLedState‘, ‘Op8CockpitLedSnapshot‘, ‘TrackedLedEntry‘, ‘DeviceLedController‘, and ‘MultiDeviceLedManager‘ + 4 comprehensive unit test suites
- ‘src/lib.rs‘ — Exported ‘pub mod led_controller‘
- ‘src/main.rs‘ — Integrated ‘MultiDeviceLedManager‘ into ‘NkEditorApp‘
- ‘js/nkeditor-sdk.js‘ — Exported ‘LED_OPERATING_MODES‘ (‘[’internal’, ’external’, ’op8_cockpit’]‘)
- ‘js/nkeditor-sdk.test.js‘ — Verified ‘LED_OPERATING_MODES‘ in the companion Node.js SDK test suite

Test coverage (34 total)

QA verdict

PASS